

Asiko TV — Revised Platform Architecture

Response to Enveu Proposal v3 (ENV/ASIKO/2026-09/03)

John Upah, Managing Director, Media Icons Africa
5 September 2026

Commercial in confidence — prepared for Enveu, not for onward distribution.

In response to	Enveu Proposal v3 — ENV/ASIKO/2026-09/03
Prepared for	Chandan Luthra, Head of Professional Services, Enveu
Prepared by	John Upah, Managing Director, Media Icons Africa
Date	5 September 2026
Service	Asiko TV — free ad-supported linear, African diaspora (UK / US / CA / EU)
Launch scope	7 channels (proof of concept) → 100+ channels (target)

1. The change, in one paragraph

We are removing AWS Elemental and AWS Elemental MediaTailor from the Asiko TV stack entirely. Server-side ad insertion moves to the Eyevinn platform, alongside the Channel Engine origin that Proposal v3 already assumes. Nothing else in your engagement changes in intent — scheduling, playout orchestration, EPG, SCTE-35 break signalling and the CMS are still the layer we need Enveu to build.

- **Why:** one platform instead of two, no AWS account or egress to manage, and every dollar not spent on infrastructure goes to content licensing — the real constraint on launch.
- **What is unaffected:** the ad *decision* server (Google Ad Manager, SpringServe or Publica) stays with Media Icons and is still integrated by Enveu. Direct sponsorship is still sold by us.
- **In parallel:** Media Icons is building a working proof of concept now, on Eyevinn Open Source Cloud, at near-zero cost. It is described in Section 4 and is offered to Enveu as the starting point for the front end and EPG rather than a greenfield build.
- **Scale is unchanged:** we accept that operating 100+ channels needs Enveu. The proof of concept proves feasibility and de-risks the origin; it does not replace the production platform.

2. The revised stack

Seven layers, one platform doing the playout and ad insertion. Read top-down as the viewer experiences it; the data flows bottom-up from the library to the screen.

LAYER	COMPONENTS	OWNER
Experience	PWA player · channel surf · EPG (now / next) · ad-break UX	POC delivers a working reference → Enveu hardens & extends
Scheduling & CMS	schedule builder · break-structure config · content metadata	Enveu builds
Ad decision	Google Ad Manager / SpringServe / Publica · direct sponsorship	Media Icons (Enveu integrates)
Ad insertion	Eyevinn SGAI Pipeline · Ad Normalizer (VAST / VMAP)	Media Icons — replaces AWS MediaTailor
Playout origin	Eyevinn Channel Engine ×7 · VOD2Live · SCTE-35 markers out	Media Icons (Enveu orchestrates)
Delivery	Bunny CDN — edge cache & egress	Media Icons
Content	transcoded HLS library · Bunny storage · Eyevinn VOD Pipeline (batch transcode)	Media Icons

Data flow: Library (Bunny) → Channel Engine (playout + SCTE-35) → Eyevinn SGAI (stitch per viewer) ⇄ Ad decision server (VAST) → Bunny CDN → PWA player.

Removed from the stack: AWS Elemental MediaLive · AWS Elemental MediaPackage · AWS Elemental MediaTailor · Hostinger KVM8 (VPS).

Component register

COMPONENT	ROLE	OWNER	STATUS VS V3
Eyevinn Channel Engine	Linear playout origin, one instance per channel (VOD2Live), emits HLS + SCTE-35	Media Icons	Unchanged
Eyevinn SGAI Pipeline	Server-side ad insertion — stitches ads against SCTE-35 markers	Media Icons	Replaces MediaTailor
Eyevinn Ad Normalizer	VAST / VMAP handling between SGAI and the ad decision server	Media Icons	New — was MediaTailor-internal
Ad decision server	Which ad plays — GAM / SpringServe / Publica	Media Icons	Unchanged
Bunny CDN	Edge cache and delivery to viewers	Media Icons	Unchanged
Bunny storage	Origin store for the transcoded HLS library	Media Icons	Unchanged
Eyevinn VOD Pipeline	Batch transcode / package to a consistent profile — run, then stop	Media Icons	Optional — or existing Bunny transcode
Scheduling & CMS	Schedule building, break-structure config, content metadata for non-engineers	Enveu build	Unchanged scope
PWA front end + EPG	Channel selection, live playback, programme guide, ad-break UX	Enveu build	Re-based on POC — extend, not build
AWS Elemental MediaTailor	Server-side ad insertion	—	Removed
AWS Elemental MediaLive / MediaPackage	Live encode / package origin	—	Removed — not required by v3 either
Hostinger KVM8	Self-managed VPS	—	Removed — Eyevinn runs the compute
Hostinger CloudStartup	Marketing / corporate site	Media Icons	Unchanged

3. What this changes against Proposal v3

Your scope in Section 2 holds almost line for line. The one substantive edit is the ad-insertion target; two front-end items re-base onto the proof of concept.

ENVEU SCOPE ITEM	REVISED POSITION
Channel scheduling & playout orchestration	Unchanged. Eyevinn Channel Engine remains the origin; your orchestration drives it from the schedule with SCTE-35 markers in place. The POC proves the origin works for 7 channels before you build on it.
SCTE-35 / break structure feeding the engine	Unchanged in effort. The markers are now consumed by Eyevinn SGA instead of MediaTailor. The schedule → break-structure → marker work is identical.
Electronic Programme Guide	Re-based. The POC already generates a working EPG (now / next / forward) from schedule JSON. Enveu formalises generation and delivery from a proven reference rather than from zero.
Ad server integration	Changed target. “Integrate with AWS MediaTailor” becomes “integrate Eyevinn SGA Pipeline + Ad Normalizer with the chosen ad decision server.” Both are SCTE-35-driven SSAI calling a VAST endpoint — like-for-like.
Viewer front end (PWA)	Re-based. The POC is a working PWA — channel surfing, HLS playback, guide, ad-break UX. Enveu hardens it to production (state backend, analytics, QA) and extends it, rather than greenfield.
Content management (CMS)	Unchanged. Not covered by the POC. Still Enveu’s to build.

On the Section 5 assumptions. Your assumptions still hold with one clarification: the shared origin both Channel Engine and the ad-insertion layer read from is Bunny storage or object storage behind Bunny CDN — **no AWS dependency**. Content is pre-transcoded HLS on one consistent profile, as you require. Media Icons runs and hosts the Channel Engine instance.

4. Where the proof of concept fits your build

The proof of concept is a Git repository — github.com/johnupah-dev/asiko-tv-poc — that Media Icons is building now and will give Enveu read access to on day one.

What it delivers today

- A working **viewer PWA**: 7 branded channels, live HLS playback, channel surfing, a programme guide, an ad-break sequence with countdown and rejoin, and a monetisation dashboard modelling Direct / PMP / Open Exchange / House demand.
- **One config file** (`data/channels.json`) holding all 7 channels, their schedules, break structure and demand weights — no code changes to re-programme.
- **Playout provisioning scripts** (`playout/`) that stand up real Eyevinn Channel Engine channels from that same config via the Open Source Cloud API, and return the live HLS playback URLs.

What it proves — so Enveu does not have to

- Eyevinn Channel Engine works as the origin for a 7-channel line-up, end to end, on real infrastructure.
- An EPG can be generated from a schedule document and rendered in the player.
- The ad-break placement and viewer experience are validated in a real player before any ad server is wired in.

- The monetisation model — avail → creative → impression → revenue, split by demand type — is expressed and can be reconciled later against the real ad server.

What stays firmly Enveu's

- Production **scheduling and playout orchestration** — the POC loops assets; it does not run a real schedule-driven linear feed with SCTE-35 emission.
- **SCTE-35 signalling** into the manifest, and the break-structure adapter feeding the engine.
- **Server-side ad insertion** wiring — Eyevinn SGAI configured per channel and connected to the ad decision server.
- A multi-user **CMS** for non-engineers; production hardening of the PWA; an analytics backend.
- Everything that makes **100+ channels** operable rather than 7.

Net effect: Enveu begins from a proven origin, a live reference front end and a working EPG, integrating one platform rather than an AWS toolchain. Two of the five Section 2 scope items shift from “build” to “extend.”

5. What we are asking Enveu to confirm

1. A revised scope, timeline and price reflecting: MediaTailor integration removed; Eyevinn SGAI + Ad Normalizer integration added; PWA front end and EPG re-based as “extend the delivered POC,” not build from zero.
2. That Eyevinn SGAI integration is a like-for-like substitution for MediaTailor integration — both SCTE-35-driven SSAI against a VAST endpoint — with no net increase to the one-time fee.
3. Day-one read access to the POC repository as the reference architecture and the starting point for the front end and EPG.
4. That the Section 5 assumptions hold with Bunny storage / object storage behind Bunny CDN as the shared origin, and no AWS component anywhere in the delivered platform.
5. The managed-operations line still stands as optional and separately quoted, not a condition of IP transfer — unchanged from v3.

6. Commercial posture

LINE	BASIS	INDICATIVE
Eyevinn — 7 channels	10 tokens / channel / day	70 / day
Eyevinn — storage, ad insertion, scheduler	platform overhead	~30 / day
Eyevinn — POC total	fits Professional plan, €69/mo, 300 tokens/day, 14-day trial	~100 / day
Eyevinn — batch transcode	250 tokens/day while running — run, then stop	one-off
Bunny CDN	usage — egress scales with audience, not channel count	variable
AWS	removed	0
Enveu — recurring	none after handover; managed ops optional	0
Enveu — build	one-time, per revised scope (v3 baseline USD 13,000)	one-time

Media Icons is a sales house, not a technology house. Capital is being held for content licensing, which is the binding constraint on launch. The revised stack exists to protect that: no AWS spend, no recurring platform fee, and a proof of concept that costs the price of a trial to stand up.

John Upah Managing Director, Media Icons Africa

Reference: response to ENV/ASIKO/2026-09/03 (v3) · 5 September 2026 · Commercial in confidence